



Lenguajes de Programación

Módulo II. Python

Tema 2: Instalación



©2026

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en

<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Windows



- Usa el instalador de <https://python.org>
- **IMPORTANTE:** Durante la instalación marca la casilla “Add Python to PATH”

MacOS



- Usa el instalador de <https://python.org>
- Alternativamente puedes instalarlo con Homebrew:

```
brew install python
```

Linux



- Normalmente viene preinstalado
 - Para verificarlo ejecuta en una terminal:
`python3 --version`
- Alternativamente puedes instalarlo en Ubuntu con (no olvides indicar que quieres el paquete python3):
`sudo apt install python3`

¿Qué se instala?



- El intérprete de Python
- La librería estándar
- El gestor de paquetes de Python Pip
 - Permite instalar paquetes adicionales que no están en la librería estándar
 - Por ejemplo: pandas para cálculo numérico



Verificación de la instalación

- Ejecuta los siguientes comandos en una terminal (en Windows el comando puede ser python y no python3):

```
patxi@patxi-EXECUTIVE-14:~/git/lp-python-2026
patxi@patxi-EXECUTIVE-14:~/git/lp-python-2026 80x8
patxi@patxi-EXECUTIVE-14 > ~/git/lp-python-2026 > main > python3 --version
Python 3.12.3
patxi@patxi-EXECUTIVE-14 > ~/git/lp-python-2026 > main > pip --version
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
patxi@patxi-EXECUTIVE-14 > ~/git/lp-python-2026 > main >
```



El problema de las dependencias

- Imagina que tienes dos proyectos Python que dependen de la librería “socket”
 - El proyecto A depende de la versión 1.0
 - El proyecto B depende de la versión 2.0
- Si instalas una determinada versión en tu sistema:
 - `pip install socket==2.0`
- El proyecto A no funcionará
- Si instalas la otra, será B el que no funcione

La mejor práctica: aislar los proyectos



- Se crea un entorno virtual para cada proyecto
 - Es simplemente una copia aislada del intérprete de Python
 - En el entorno virtual puedes instalar paquetes sin afectar a otros proyectos
- Para ello se utiliza el módulo “venv”



Uso de venv

- Creación del entorno:
 - Nos situamos en la carpeta donde tengamos el proyecto
 - En una terminal creamos el entorno:
 - `python3 -m venv mi-entorno`
- Activamos el entorno para trabajar en él (mientras no lo activemos los paquetes se siguen instalando globalmente o en el entorno que esté activado):
 - **Win:**
 - `mi-entorno\Scripts\activate.bat`
 - **MacOS/Linux:**
 - `source mi-entorno/bin/activate`



Uso de venv

venv crea una carpeta con el nombre del entorno

```
patxi@patxi-EXECUTIVE-14 ~/tmp/pruebas-python$ python3 -m venv mi-entorno
patxi@patxi-EXECUTIVE-14 ~/tmp/pruebas-python$ ll
total 4,0K
drwxrwxr-x 5 patxi patxi 4,0K ene 22 11:27 mi-entorno
patxi@patxi-EXECUTIVE-14 ~/tmp/pruebas-python$ source mi-entorno/bin/activate
(mi-entorno) patxi@patxi-EXECUTIVE-14 ~/tmp/pruebas-python$
```

En la shell se nos indica que estamos en el entorno "mi-entorno"



Uso de venv

- Desactivación del entorno:
 - En la carpeta del proyecto donde hemos creado el entorno ejecutamos:

`deactivate`



Instalación de paquetes

- Python dispone de infinidad de paquetes desarrollados por terceros
- Pip instala los paquetes por defecto de **PyPI**(Python Package Index)
- Si no tenemos un entorno activo los instalará de forma global en el sistema
 - Desaconsejado, puede haber conflictos de dependencias
- Si tenemos un entorno virtual activo, los instalará en dicho entorno
- Sintaxis:

```
pip install paquete
```

- Por ejemplo:

```
pip install Django
```



Instalación de paquetes

- El comando anterior instala por defecto la última versión estable del paquete
- Esto no siempre es lo que queremos
 - Nuestro proyecto puede estar usando una versión más antigua
 - La última versión tiene alguna incompatibilidad con funcionalidades que ya estábamos usando
 - Por fallos de seguridad aún no resueltos
- Para instalar una versión específica:

```
pip install paquete==version
```

- Por ejemplo (la última versión de Django es la 6.0.1):

```
pip install Django==5.2.10
```



Instalación de paquetes

- No especificar la versión al instalar paquetes es en general una **mala práctica**:
 - No sabemos qué versión se está instalando
 - No sabemos si esta versión es compatible con la que estábamos usando
 - Si se publica una nueva versión, se instalará esta nueva en lugar de la que teníamos en un principio
 - No es reproducible la instalación
- Es preferible siempre especificar la versión al instalar
- Para que la instalación de paquetes sea más reproducible, se pueden listar todos ellos en un fichero requirements.txt e instalarlos todos a la vez con

```
pip install -r requirements.txt
```